Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили:
студенты группы 25ВВВ4
Исянов Д.Ф.
Качкуркин Р.Н.

Приняли:
Юрова О.В.,
Деев М.В.

Пенза 2026

1. Цель работы

Изучение динамических структур данных языка Си на примере односвязных линейных списков, приобретение практических навыков работы с указателями, динамическим выделением и освобождением памяти (malloc, free), а также реализация базовых операций над списками (добавление, удаление, просмотр, поиск элементов и проверка на дубликаты) и построение на их основе классических структур данных: стека (LIFO), очереди (FIFO) и приоритетной очереди.

2. Задания

Задание 1: Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Задание 2: На основе приведенного кода реализуйте структуру данных Очередь.

Задание 3: На основе приведенного кода реализуйте структуру данных Стек.

3. Пояснительный текст к программам

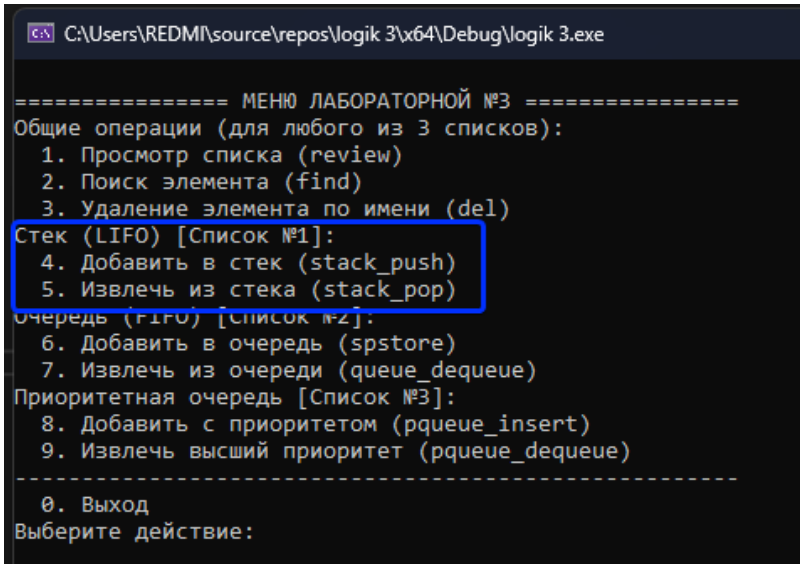
Задание 1

В программе объявляется отдельный односвязный список для приоритетной очереди с указателями pqueue_head и pqueue_last. Для каждого узла структуры struct node задаются имя объекта (inf) и числовое значение приоритета (priority).

При добавлении элемента функцией pqueue_insert проверяется уникальность имени. Если список пуст или приоритет нового элемента выше, чем у первого, узел ставится в начало списка. В противном случае выполняется перебор списка до тех пор, пока приоритет следующего элемента больше либо равен новому, после чего узел вставляется на найденную позицию.

Для извлечения элемента функция pqueue_dequeue считывает данные из головы списка (элемент с наибольшим приоритетом), сдвигает указатель pqueue_head на следующий узел и освобождает память через free.

Результат работы программы:



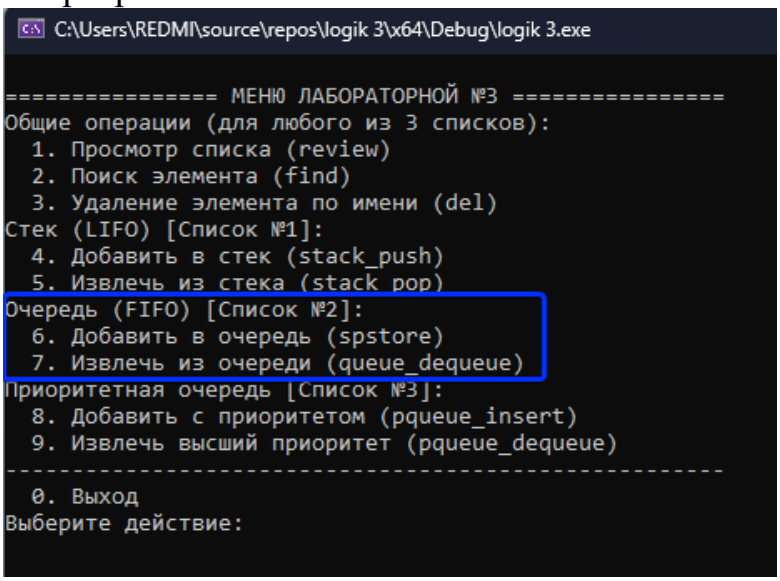
Задание 2

В программе организуется структура данных «Очередь» (FIFO) с собственными указателями queue_head и queue_last.

Добавление узла выполняется функцией spstore: выделяется память под структуру, проверяется уникальность имени, а новый элемент присоединяется к концу списка (last->next = p), после чего указатель конца сдвигается на него (last = p).

Извлечение реализуется функцией queue_dequeue: программа считывает данные из начала очереди (queue_head), сдвигает указатель головы на следующий элемент и очищает память извлеченного узла вызовом free.

Результат работы программы:



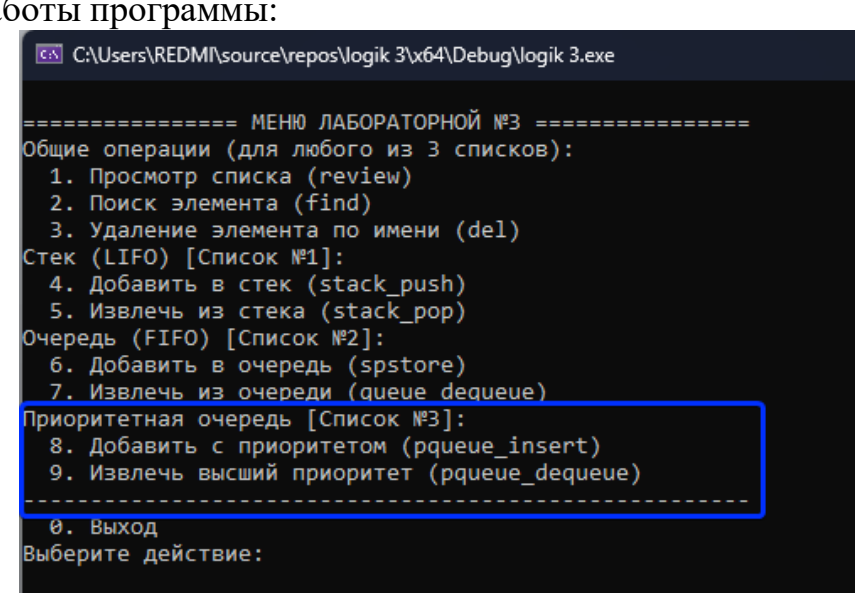
Задание 3

В программе реализуется структура данных «Стек» (LIFO) на основе указателя на вершину stack_head.

Добавление элемента (Push) выполняется функцией stack_push: после проверки на дубликаты новый узел связывается с текущей вершиной (p->next = stack_head), и указатель вершины перемещается на созданный элемент.

Извлечение элемента (Pop) реализовано функцией stack_pop: программа выводит значение верхнего узла, переносит указатель вершины на следующий элемент цепи (stack_head = stack_head->next) и удаляет извлеченный узел из памяти с помощью free.

Результат работы программы:



4. Псевдокод

Задание 1

Алгоритм добавления с приоритетом (pqueue_insert):

```
АЛГОРИТМ Добавить_В_Приоритетную_Очередь:
    Выделить память под новый узел p
    Ввести имя объекта name
    ЕСЛИ name уже существует в очереди ТО:
        Вывести "Ошибка: элемент уже есть"
        Освободить память p
        ВЫХОД
    КОНЕЦ ЕСЛИ

    Ввести числовое значение priority
    Установить p.inf = name, p.priority = priority, p.next = NULL

    ЕСЛИ pqueue_head == NULL ИЛИ p.priority > pqueue_head.priority
ТО:
        p.next = pqueue_head
        pqueue_head = p
        ЕСЛИ pqueue_last == NULL ТО pqueue_last = p
    ИНАЧЕ:
        curr = pqueue_head
        ПОКА curr.next != NULL И curr.next.priority >= p.priority
ДЕЛАТЬ:
            curr = curr.next
        КОНЕЦ ПОКА
        p.next = curr.next
        curr.next = p
        ЕСЛИ p.next == NULL ТО pqueue_last = p
    КОНЕЦ ЕСЛИ
КОНЕЦ АЛГОРИТМА
```

Алгоритм извлечения (pqueue_dequeue):

```
АЛГОРИТМ Извлечь_Высший_Приоритет:
    ЕСЛИ pqueue_head == NULL ТО:
        Вывести "Приоритетная очередь пуста"
        ВЫХОД
    КОНЕЦ ЕСЛИ

    temp = pqueue_head
    Вывести temp.inf и temp.priority
    pqueue_head = pqueue_head.next
    ЕСЛИ pqueue_head == NULL ТО pqueue_last = NULL
    Освободить память temp
КОНЕЦ АЛГОРИТМА
```

Задание 2

Алгоритм добавления в конец (spstore):

```
АЛГОРИТМ Добавить_В_Очередь:
    Создать новый узел p через get_struct() (с проверкой на
уникальность)
    ЕСЛИ p == NULL ТО:
        ВЫХОД
    КОНЕЦ ЕСЛИ

    ЕСЛИ queue_head == NULL ТО:
        queue_head = p
        queue_last = p
    ИНАЧЕ:
        queue_last.next = p
        queue_last = p
    КОНЕЦ ЕСЛИ
КОНЕЦ АЛГОРИТМА
```

Алгоритм извлечения из начала (queue_dequeue):

```
АЛГОРИТМ Извлечь_Из_Очереди:
    ЕСЛИ queue_head == NULL ТО:
        Вывести "Очередь пуста"
        ВЫХОД
    КОНЕЦ ЕСЛИ

    temp = queue_head
    Вывести temp.inf
    queue_head = queue_head.next
    ЕСЛИ queue_head == NULL ТО queue_last = NULL
    Освободить память temp
КОНЕЦ АЛГОРИТМА
```

Задание 3

Алгоритм добавления на вершину (stack_push):

```
АЛГОРИТМ Поместить_В_Стек:
    Создать новый узел p через get_struct() (с проверкой на
уникальность)
    ЕСЛИ p == NULL ТО:
        ВЫХОД
    КОНЕЦ ЕСЛИ

    p.next = stack_head
    stack_head = p
    ЕСЛИ stack_last == NULL ТО stack_last = p
КОНЕЦ АЛГОРИТМА
```

Алгоритм извлечения с вершины (stack_pop):

```
АЛГОРИТМ Извлечь_Из_Стека:
    ЕСЛИ stack_head == NULL ТО:
        Вывести "Стек пуст"
        ВЫХОД
    КОНЕЦ ЕСЛИ
```

```
    temp = stack_head
    Вывести temp.inf
    stack_head = stack_head.next
    ЕСЛИ stack_head == NULL ТО stack_last = NULL
    Освободить память temp
КОНЕЦ АЛГОРИТМА
```

5. Вывод

Изучили и закрепили основные приёмы работы с динамическими структурами данных языка Си, а именно односвязными линейными списками, динамическим выделением и освобождением памяти с помощью функций malloc и free, а также научились строить на их основе структуры данных стек, очередь и приоритетная очередь, выполняя вставку, извлечение, поиск, удаление элементов и контроль уникальности данных.